

1. Problem Statement

Design a load balancer for an e-commerce platform serving traffic across a pool of 8 application servers. The platform currently suffers from uneven traffic distribution — individual servers receive disproportionate load, causing cascading failures during peak traffic periods. Manual detection of server failures takes 15-20 minutes, during which failed servers continue receiving traffic and returning errors to users.

The load balancer must distribute traffic intelligently across healthy servers, detect and respond to server failures automatically within seconds, handle peak traffic of 50,000 requests per second, and add no more than 5ms of overhead to request processing time.

1.1 Functional Requirements

- Distribute incoming requests across 8 application servers using an appropriate algorithm.
- Detect unhealthy or crashed servers automatically and remove them from the active pool.
- Re-add recovered servers to the pool gradually after confirming their health.
- Handle peak traffic of 50,000 requests per second without server crashes.
- Maintain high availability — the load balancer itself must not be a single point of failure.

1.2 Non-Functional Requirements

- **Latency overhead:** The load balancer must add no more than 5ms to request processing time.
- **Detection speed:** Server failure must be detected and acted upon within 30 seconds — compared to the current 15-20 minute manual process.
- **Availability priority:** The platform prioritises availability over consistency — users should always receive a response, even if occasionally from a slightly slower server.
- **Scale:** Normal load: ~10,000 req/sec. Peak load: ~50,000 req/sec (Black Friday, flash sales) — spikes can be sudden, not gradual ramps.

1.3 Out of Scope

- Multi-region or global load balancing — the platform currently operates in a single country (India).
- Auto-scaling of the application server pool — the 8 servers are fixed for this version.
- SSL termination, DDoS protection, or WAF functionality.

2. Capacity Estimation

Metric	Value	Implication
Normal traffic	10,000 req/sec	Load balancer decision overhead must stay well under 5ms even at sustained load
Peak traffic	50,000 req/sec	5x normal — spikes can be sudden; algorithm must handle burst without reconfiguration
Application servers	8 servers	At peak: ~6,250 req/sec per server average; distribution algorithm must prevent any single server from exceeding capacity
Current failure detection	15–20 minutes (manual)	Target: ≤30 seconds via automated health checks
Max load balancer overhead	5ms per request	Eliminates any algorithm requiring per-request database queries, timestamp scans, or complex statistical computation

Key implication

The current failure mode is a cascading crash: uneven distribution causes one or two servers to hit capacity and crash, which pushes their traffic onto the remaining servers, triggering further crashes. The load balancer must solve both the distribution problem (Least Connections) and the detection problem (automated health checks) to prevent this cascade.

3. Algorithm Selection

Four algorithms were evaluated against the specific constraints: 50,000 req/sec peak throughput, mixed request types (lightweight browsing vs. expensive checkout), and a hard 5ms overhead ceiling.

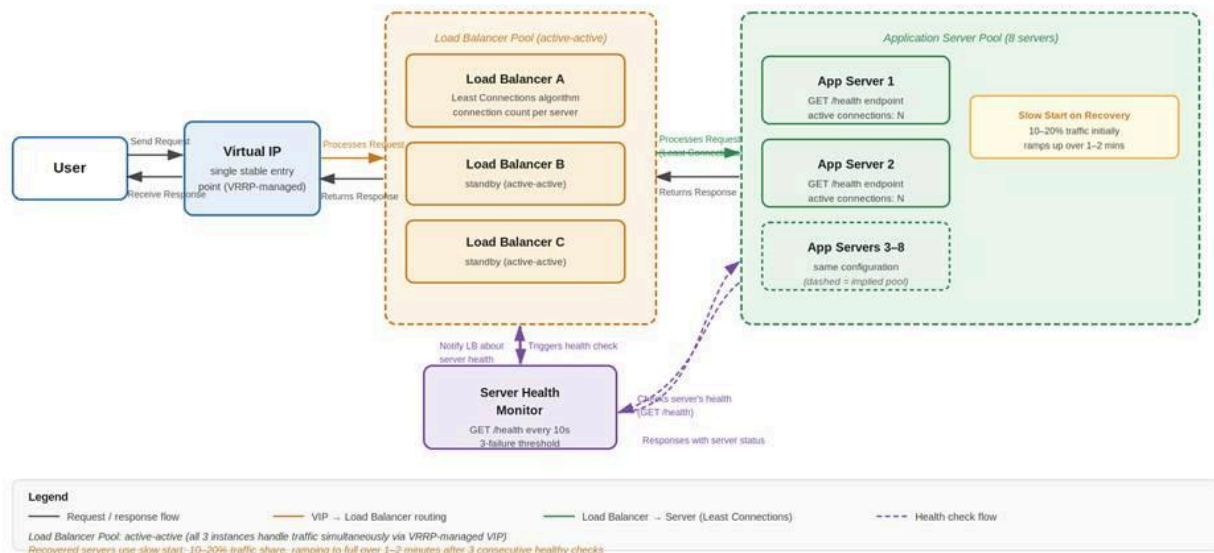
Algorithm	How it works	Verdict for this system
Round Robin	Cycles through servers in fixed order regardless of load	Rejected — distributes requests evenly but not load. A server handling 3 checkout requests (2s each) and one handling 3 browsing requests (50ms each) receive the same number of new requests, causing the imbalance the stakeholder reported.

Least Response Time	Routes to server with fewest connections AND lowest recent average response time	Viable but over-engineered — requires continuous per-server response time measurement and rolling averages at 50,000 req/sec. This computation itself risks breaching the 5ms overhead constraint.
Weighted Round Robin	Round Robin with capacity weighting per server	Rejected — assumes static server capacities. Does not adapt to runtime load variations or mixed request types.
Least Connections	Routes to server with fewest currently active connections	Selected — $O(1)$ decision per request (single integer comparison per server). Self-corrects naturally: expensive requests stay active longer, so servers handling them accumulate higher counts and receive fewer new requests until those complete.

3.1 Why Least Connections Self-Corrects for Mixed Request Types

A potential objection: if Server A has 3 checkout requests (each taking 2 seconds) and Server B has 3 browsing requests (each taking 50ms), both show 3 active connections — Least Connections might route the next request to either. However, Server B's connections clear in ~50ms, dropping its count to near 0, while Server A's count stays at 3 for nearly 2 seconds. The next several requests route to Server B automatically. The algorithm self-corrects within one request cycle — at 50,000 req/sec, this is a negligible window. The bounded, self-healing nature of this imbalance does not justify the guaranteed continuous computation cost of Least Response Time.

4. High-Level Architecture



4.1 Components

Component	Responsibility
Virtual IP (VIP)	A single, stable IP address exposed to users and DNS. Managed by VRRP across all load balancer instances — if one instance goes down, another claims the VIP within seconds, transparent to users whose traffic is always directed at the same IP.
Load Balancer Pool (A, B, C)	Three load balancer instances running in active-active configuration — all three handle traffic simultaneously. Each instance independently applies the Least Connections algorithm and maintains its own view of active connection counts per server.
Server Health Monitor	A component within each load balancer instance that triggers periodic health checks (GET /health) against every application server. Maintains failure and success counts per server and notifies the load balancer to remove or re-add servers based on thresholds.
Application Server Pool	8 application servers handling actual business logic (browsing, cart, checkout). Each exposes a GET /health endpoint returning HTTP 200 when ready and non-200 when degraded. Active connection count is tracked per server by the load balancer.

5. Health Monitoring

The health monitoring system directly addresses the stakeholder's core pain point: 15-20 minute manual failure detection. The automated system detects failure within approximately 30 seconds and acts without human intervention.

5.1 Health Check Mechanism

The Server Health Monitor sends an HTTP GET request to the /health endpoint of each application server every 10 seconds. An HTTP health check was chosen over a raw network ping (ICMP) specifically because:

- A network ping confirms the server's network interface is alive — but the application could be crashed, deadlocked, or out of memory while still responding to pings.
- GET /health tests whether the application itself is ready to handle traffic — a genuine application-level health signal.

Each application server implements a lightweight /health endpoint that returns HTTP 200 when the application is healthy and ready, and HTTP 5xx when it is degraded, starting up, or shutting down.

5.2 Failure Detection Thresholds

Parameter	Value	Reasoning
Health check interval	Every 10 seconds	Checking every second across 8 servers creates 8 competing health check connections per second — unnecessary load. Every 10 seconds balances detection speed against overhead.
Failure threshold	3 consecutive failed checks	A single missed check could be a transient network blip or GC pause — not a real failure. 3 consecutive misses is a strong signal of genuine failure.
Worst-case detection time	~30 seconds	3 failures × 10 second interval = 30 seconds. Compared to 15-20 minute manual detection — a 30-60x improvement.
Action on failure	Remove server from active pool immediately	No more traffic routed to failed server. Remaining servers absorb the load.

5.3 Server Recovery — Slow Start

When a previously failed server begins returning healthy responses, it is not immediately restored to full traffic share. The recovery process:

1. Server Health Monitor detects 3 consecutive successful health check responses.
2. Server is re-added to the active pool at 10-20% of a normal server's traffic share.
3. Load balancer gradually increases the server's traffic weight over 1-2 minutes as it continues to handle requests successfully.
4. After the ramp-up period, the server receives its full equal share of traffic.
5. If the server fails again during ramp-up, it is immediately removed and the slow start process resets.

Why slow start matters

A server that just crashed may have cold caches, incompletely initialized connection pools, or unresolved underlying resource pressure. Immediately routing full traffic share risks triggering the same crash again within seconds of recovery — causing the server to flap in and out of the pool repeatedly, making the situation worse than simply leaving it out. Slow start gives the server time to stabilise under light load before bearing full traffic.

6. High Availability — Load Balancer Pool

Without redundancy, the load balancer itself is a single point of failure — if it goes down, all 8 application servers become unreachable even if they are perfectly healthy. Three load balancer instances running in active-active configuration via VRRP eliminate this risk.

6.1 Active-Active vs Active-Passive

Model	How it works	Trade-off
Active-Passive	One primary load balancer handles all traffic; a standby instance waits idle and takes over on primary failure	Failover gap between primary failure detection and standby activation (~10-30 seconds of downtime). Standby capacity is wasted during normal operation.
Active-Active (selected)	All three instances handle traffic simultaneously. If one fails, the other two continue without interruption — no failover required	All capacity is used at all times. No failover gap. Slightly more complex to manage shared state (though load balancers are largely stateless, making this manageable).

6.2 Virtual IP and VRRP

Users and DNS are pointed at a single Virtual IP (VIP) rather than any individual load balancer's IP address. VRRP (Virtual Router Redundancy Protocol) manages which physical load balancer instance currently 'owns' the VIP. If the owning instance goes down, another instance claims the VIP within seconds — transparent to users because the IP address they're connecting to never changes.

Alternative approaches considered:

- **DNS-based load balancing:** DNS returns multiple IPs (one per LB instance). Rejected — DNS TTL caching means clients keep hitting a failed instance's IP for minutes or hours after failure.
- **Cloud provider managed load balancer:** Viable if the platform is cloud-hosted; the provider handles VIP, VRRP, and HA automatically. Noted as the preferred approach if migrating to cloud — avoids managing VRRP manually.
- **Anycast routing:** Multiple instances share the same IP at the BGP routing level. Rejected — requires BGP control, appropriate for CDN-scale deployments, overkill for single-country e-commerce.

7. Request Flow

7.1 Happy Path — Server Available

6. User sends request to the Virtual IP address.
7. VIP routes the request to one of the active load balancer instances (A, B, or C).
8. Load balancer queries active connection count for each server in the pool.
9. Load balancer selects the server with the fewest active connections (Least Connections).
10. Load balancer forwards the request to the selected application server, incrementing its active connection count.
11. Application server processes the request and returns a response to the load balancer.
12. Load balancer decrements the server's active connection count and forwards the response to the VIP.
13. VIP returns the response to the user.

7.2 Unhappy Path — Server Failure

14. Server Health Monitor detects 3 consecutive failed GET /health responses from App Server N over ~30 seconds.
15. Server Health Monitor notifies the load balancer: App Server N is unhealthy.

16. Load balancer removes App Server N from the active pool immediately — no new requests are routed to it.
17. Traffic redistributes across remaining healthy servers using Least Connections.
18. Server Health Monitor continues checking App Server N every 10 seconds.
19. When App Server N returns 3 consecutive healthy responses, slow start begins — 10-20% traffic share, ramping to full over 1-2 minutes.

7.3 Unhappy Path — Load Balancer Failure

20. One load balancer instance (e.g. Load Balancer A) goes down.
21. VRRP detects the failure within seconds and reassigns the VIP to Load Balancer B or C.
22. Traffic continues flowing to the application server pool through the remaining instances — no user-visible interruption.
23. The failed load balancer instance is restarted and re-joins the active-active pool.

8. Summary and Design Principles

This design resolves two distinct problems the stakeholder reported — uneven distribution causing cascading crashes, and 15-20 minute manual failure detection — through two targeted architectural choices:

- **Least Connections over Round Robin:** addresses uneven distribution by routing based on actual server load (active connections) rather than position in a fixed cycle. Self-corrects naturally for mixed request types without requiring response time measurement or complex computation — keeping the load balancer well within its 5ms overhead budget.
- **Automated health checks over manual detection:** reduces failure detection from 15-20 minutes to ~30 seconds through periodic GET /health checks with a 3-consecutive-failure threshold, eliminating both the false-positive risk of single-check reactions and the human latency of manual monitoring.
- **Slow start on recovery:** prevents the flapping failure mode where a recovered server crashes immediately under full traffic, by introducing it at 10-20% capacity and ramping up gradually over 1-2 minutes.
- **Active-active load balancer pool via VRRP:** eliminates the load balancer as a single point of failure without the idle-capacity waste of active-passive, at no additional detection latency since no failover is required when all instances are always active.